

# CS60021: Scalable Data Mining

## DL Frameworks

Sourangshu Bhattacharya

# CPU VS GPU

Slides taken from:

Fei-Fei Li & Justin Johnson & Serena Yeung, Stanford University

# Spot the CPU!

(central processing unit)



This image is licensed under [CC-BY 2.0](#)



# Spot the GPUs!

(graphics processing unit)



This image is in the public domain



# CPU vs GPU

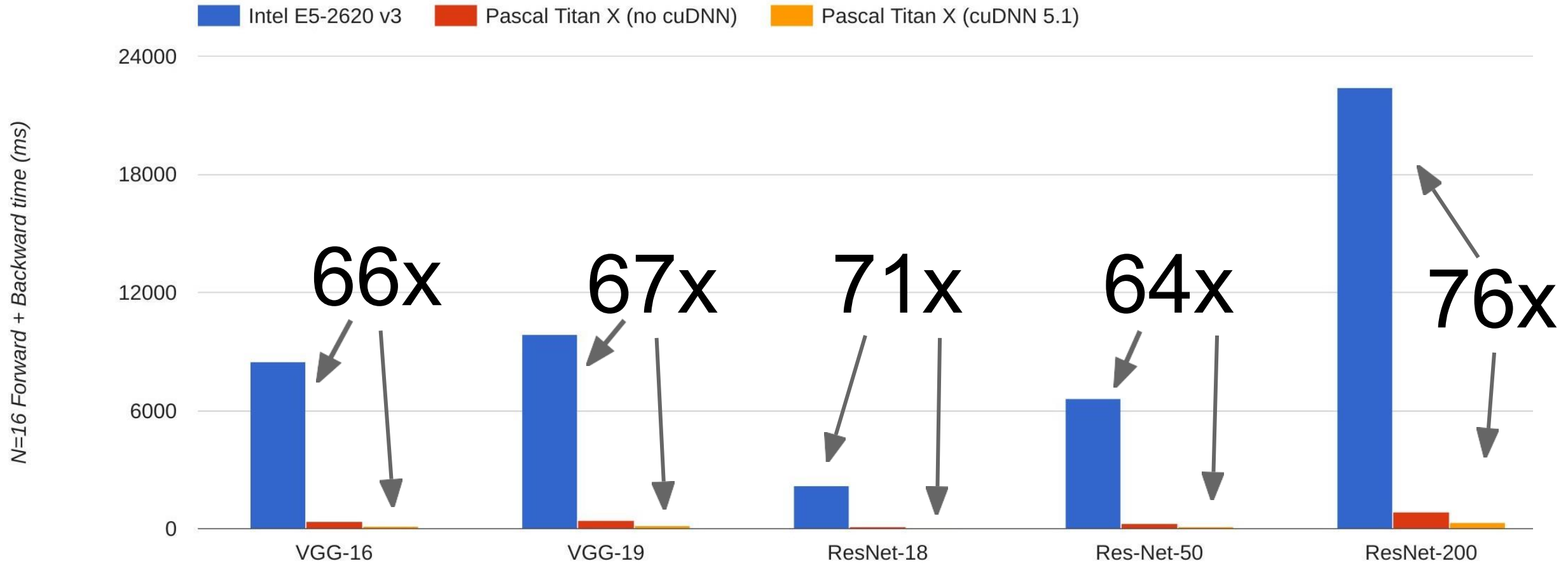
|                                     | # Cores                                | Clock Speed | Memory             | Price  |
|-------------------------------------|----------------------------------------|-------------|--------------------|--------|
| <b>CPU</b><br>(Intel Core i7-7700k) | 4<br>(8 threads with hyperthreading)   | 4.4 GHz     | Shared with system | \$339  |
| <b>CPU</b><br>(Intel Core i7-6950X) | 10<br>(20 threads with hyperthreading) | 3.5 GHz     | Shared with system | \$1723 |
| <b>GPU</b><br>(NVIDIA Titan Xp)     | 3840                                   | 1.6 GHz     | 12 GB GDDR5X       | \$1200 |
| <b>GPU</b><br>(NVIDIA GTX 1070)     | 1920                                   | 1.68 GHz    | 8 GB GDDR5         | \$399  |

**CPU:** Fewer cores, but each core is much faster and much more capable; great at sequential tasks

**GPU:** More cores, but each core is much slower and “dumber”; great for parallel tasks

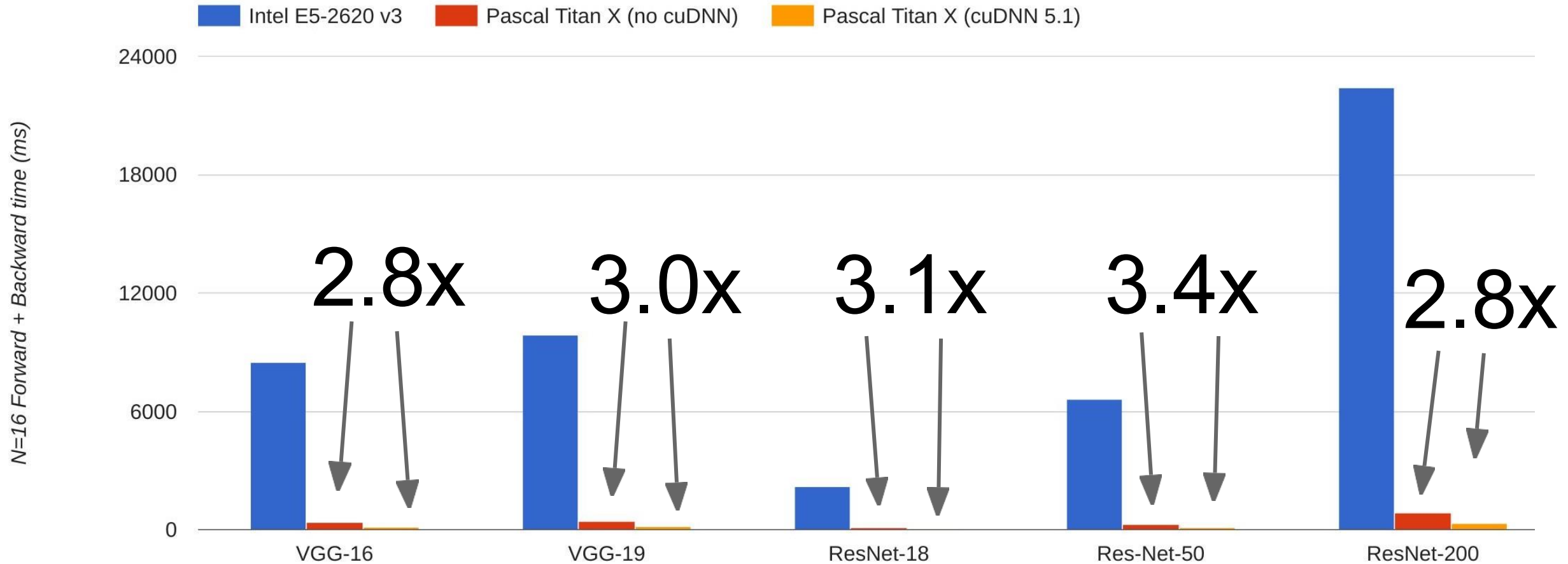
# CPU vs GPU in practice

(CPU performance not well-optimized, a little unfair)



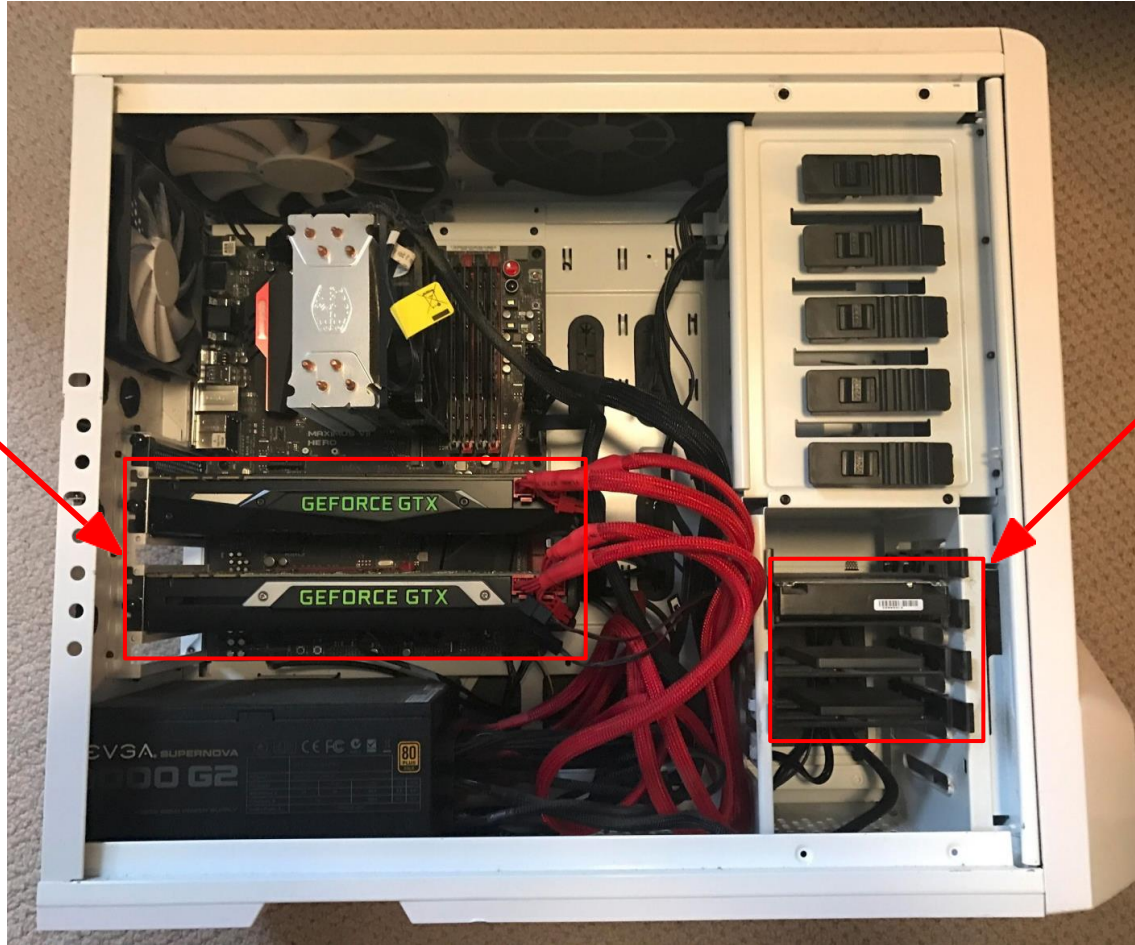
# CPU vs GPU in practice

cuDNN much faster than  
“unoptimized” CUDA



# CPU / GPU Communication

Model  
is here



Data is here

If you aren't careful, training can bottleneck on reading data and transferring to GPU!

## **Solutions:**

- Read all data into RAM
- Use SSD instead of HDD
- Use multiple CPU threads to prefetch data

# DEEP LEARNING FRAMEWORKS

Slides taken from:

Fei-Fei Li & Justin Johnson & Serena Yeung, Stanford University

# Major DL Frameworks Today

Caffe  
(UC Berkeley)



Caffe2  
(Facebook)

Torch  
(NYU / Facebook)



PyTorch  
(Facebook)

Theano  
(U Montreal)



TensorFlow  
(Google)

Mostly these

Paddle  
(Baidu)

CNTK  
(Microsoft)

MXNet  
(Amazon)

Developed by U Washington, CMU, MIT,  
Hong Kong U, etc but main framework of  
choice at AWS

And others...

# The point of deep learning frameworks

- (1) Easily build big computational graphs
- (2) Easily compute gradients in computational graphs
- (3) Run it all efficiently on GPU (wrap cuDNN, cuBLAS, etc)

# Computational Graphs

## Numpy

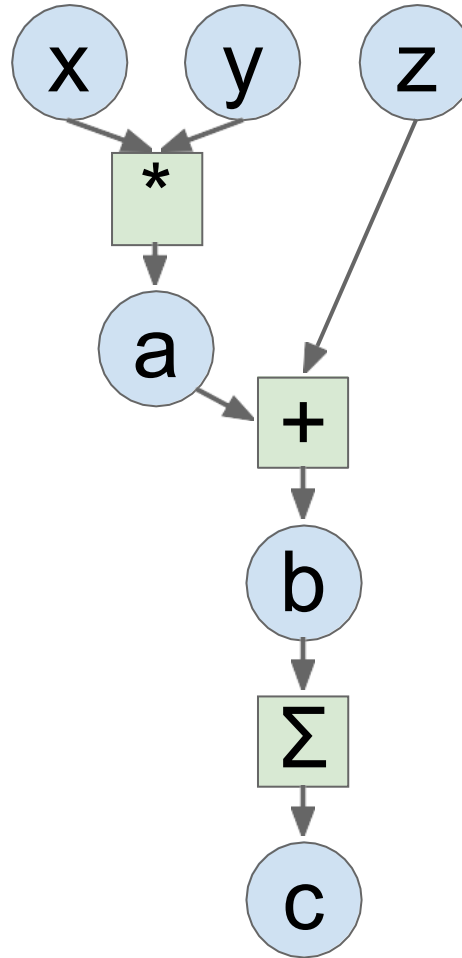
```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

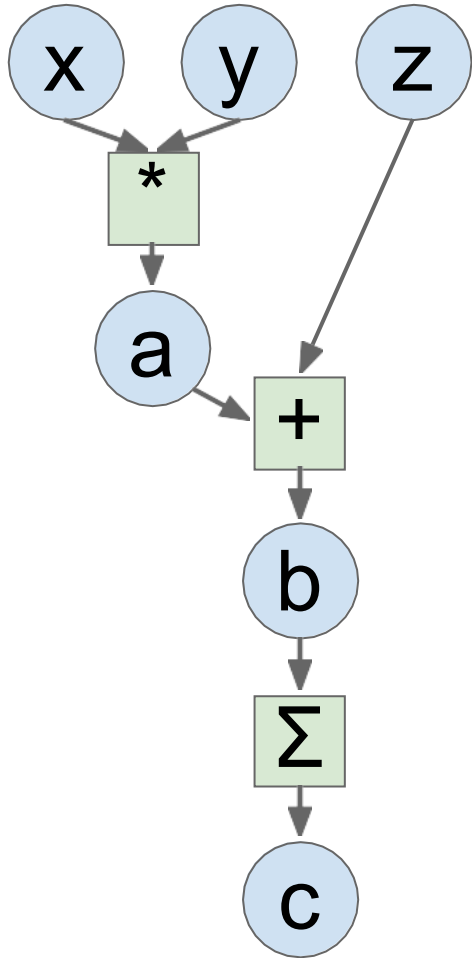
grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```



## Problems:

- Can't run on GPU
- Have to compute our own gradients

# Computational Graphs



# PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

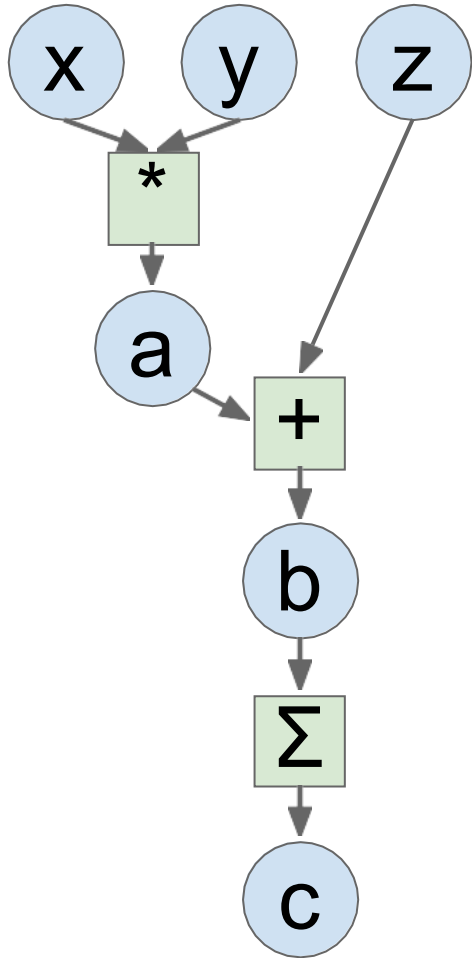
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

# Computational Graphs



Define **Variables** to start building a computational graph

## PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

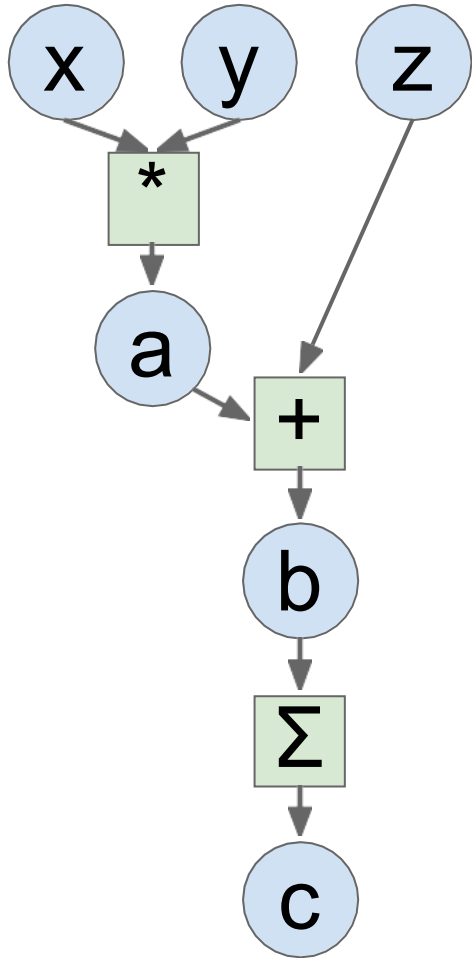
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

# Computational Graphs



Forward pass  
looks just like numpy

## PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

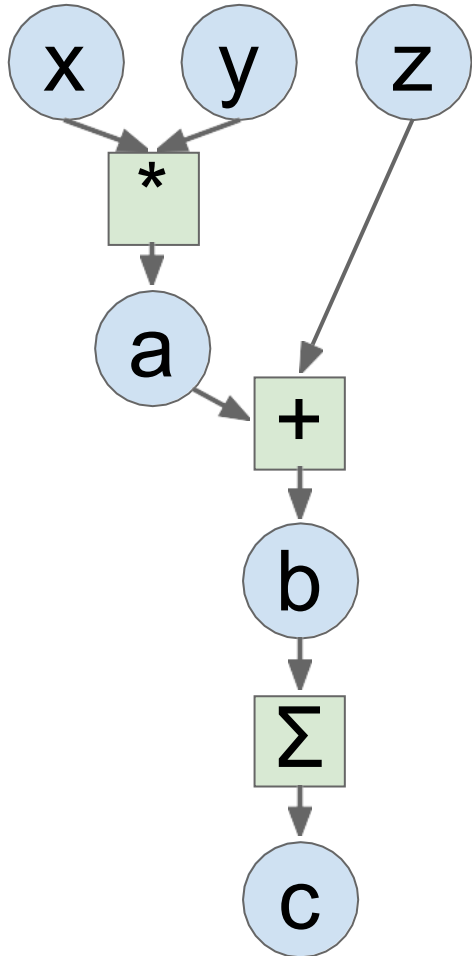
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

# Computational Graphs



Calling `c.backward()`  
computes all gradients

## PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

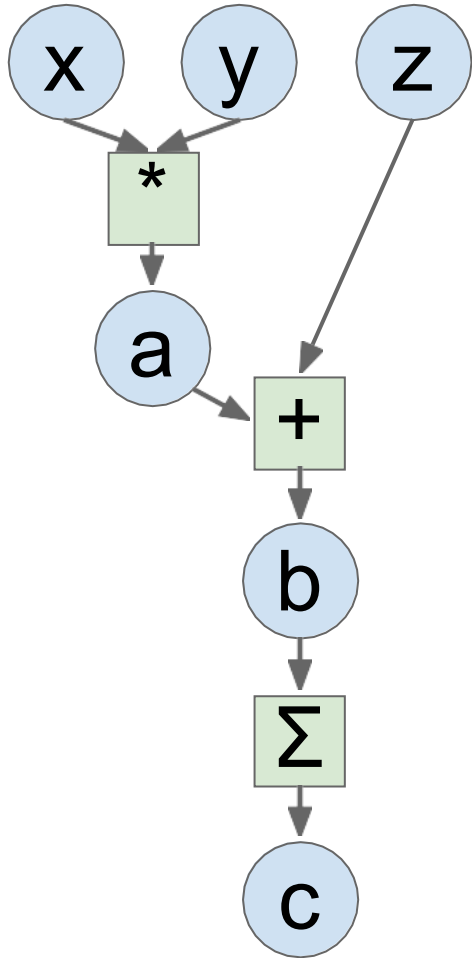
x = Variable(torch.randn(N, D),
              requires_grad=True)
y = Variable(torch.randn(N, D),
              requires_grad=True)
z = Variable(torch.randn(N, D),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

# Computational Graphs



Run on GPU by  
casting to .cuda()

# PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

x = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
y = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
z = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

# Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

# PyTorch

```
import torch
from torch.autograd import Variable

N, D = 3, 4

x = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
y = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)
z = Variable(torch.randn(N, D).cuda(),
              requires_grad=True)

a = x * y
b = a + z
c = torch.sum(b)

c.backward()

print(x.grad.data)
print(y.grad.data)
print(z.grad.data)
```

PyTorch (Autograd)

# PyTorch: Levels of Abstraction

- **Tensor:** Imperative ndarray, but runs on GPU
- **Variable:** Node in a computational graph; stores data and gradient
- These are now combined. Any tensor becomes part of the computation graph if they have (`requires_grad = "True"`)
- **Module:** A neural network layer; may store state or learnable weights

# PyTorch: Tensors

PyTorch Tensors are just like numpy arrays, but they can run on GPU.

No built-in notion of computational graph, or gradients, or deep learning.

Here we fit a two-layer net using PyTorch Tensors:

```
import torch
```

```
dtype = torch.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu.mm(w2)
```

```
    loss = (y_pred - y).pow(2).sum()
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t().mm(grad_y_pred)
```

```
    grad_h_relu = grad_y_pred.mm(w2.t())
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t().mm(grad_h)
```

```
    w1 -= learning_rate * grad_w1
```

```
    w2 -= learning_rate * grad_w2
```

# PyTorch: Tensors

Create random tensors  
for data and weights

```
import torch
```

```
dtype = torch.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu.mm(w2)
```

```
    loss = (y_pred - y).pow(2).sum()
```

```
    grad_y_pred = 2.0 * (y_pred - y)
```

```
    grad_w2 = h_relu.t().mm(grad_y_pred)
```

```
    grad_h_relu = grad_y_pred.mm(w2.t())
```

```
    grad_h = grad_h_relu.clone()
```

```
    grad_h[h < 0] = 0
```

```
    grad_w1 = x.t().mm(grad_h)
```

```
    w1 -= learning_rate * grad_w1
```

```
    w2 -= learning_rate * grad_w2
```

# PyTorch: Tensors

Forward pass: compute predictions and loss

```
import torch
```

```
dtype = torch.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()
```

```
grad_y_pred = 2.0 * (y_pred - y)
grad_w2 = h_relu.t().mm(grad_y_pred)
grad_h_relu = grad_y_pred.mm(w2.t())
grad_h = grad_h_relu.clone()
grad_h[h < 0] = 0
grad_w1 = x.t().mm(grad_h)
```

```
w1 -= learning_rate * grad_w1
w2 -= learning_rate * grad_w2
```

# PyTorch: Tensors

Backward pass:  
manually compute  
gradients

```
import torch
```

```
dtype = torch.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
```

```
for t in range(500):
```

```
    h = x.mm(w1)
```

```
    h_relu = h.clamp(min=0)
```

```
    y_pred = h_relu.mm(w2)
```

```
    loss = (y_pred - y).pow(2).sum()
```

```
    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)
```

```
w1 -= learning_rate * grad_w1
```

```
w2 -= learning_rate * grad_w2
```

# PyTorch: Tensors

Gradient descent  
step on weights

```
import torch

dtype = torch.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

# PyTorch: Tensors

To run on GPU, just cast tensors to a cuda datatype!

```
import torch
```

```
dtype = torch.cuda.FloatTensor
```

```
N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)
```

```
learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

# PyTorch: Autograd

A PyTorch **Variable** is a node in a computational graph

x.data is a Tensor

x.grad is a Variable of gradients  
(same shape as x.data)

x.grad.data is a Tensor of gradients

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

# PyTorch: Autograd

PyTorch Tensors and Variables  
have the same API!

Variables remember how they were  
created (for backprop)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

# PyTorch: Autograd

We will not want gradients  
(of loss) with respect to data

Do want gradients with  
respect to weights

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

# PyTorch: Autograd

Forward pass looks exactly the same as the Tensor version, but everything is a variable now

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

# PyTorch: Autograd

Compute gradient of loss  
with respect to w1 and w2 (zero out  
grads first)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

# PyTorch: Autograd

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()
```

Make gradient  
step on weights



```
w1.data -= learning_rate * w1.grad.data
w2.data -= learning_rate * w2.grad.data
```

# PyTorch: New Autograd Functions

Define your own autograd functions by writing forward and backward for Tensors

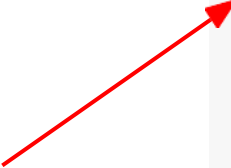
(similar to modular layers in A2)

```
class ReLU(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x):  
        ctx.save_for_backward(x)  
        return x.clamp(min=0)  
  
    @staticmethod  
    def backward(ctx, grad_y):  
        x, = ctx.saved_tensors  
        grad_input = grad_y.clone()  
        grad_input[x < 0] = 0  
        return grad_input
```

# PyTorch: New Autograd Functions

```
class ReLU(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x):  
        ctx.save_for_backward(x)  
        return x.clamp(min=0)  
  
    @staticmethod  
    def backward(ctx, grad_y):  
        x, = ctx.saved_tensors  
        grad_input = grad_y.clone()  
        grad_input[x < 0] = 0  
        return grad_input
```

Can use our new autograd  
function in the forward pass



```
N, D_in, H, D_out = 64, 1000, 100, 10  
  
x = Variable(torch.randn(N, D_in), requires_grad=False)  
y = Variable(torch.randn(N, D_out), requires_grad=False)  
w1 = Variable(torch.randn(D_in, H), requires_grad=True)  
w2 = Variable(torch.randn(H, D_out), requires_grad=True)  
  
learning_rate = 1e-6  
for t in range(500):  
    relu = ReLU()  
    y_pred = relu(x.mm(w1)).mm(w2)  
    loss = (y_pred - y).pow(2).sum()  
  
    if w1.grad: w1.grad.data.zero_()  
    if w2.grad: w2.grad.data.zero_()  
    loss.backward()  
  
    w1.data -= learning_rate * w1.grad.data  
    w2.data -= learning_rate * w2.grad.data
```

# Automatic Differentiation

*What backward() is actually computing, underneath the framework*

## Scalar case

### Forward pass

$$z = wx + b$$

$$L = (z - y)^2$$

### Backward pass

(chain rule)

$$\frac{\partial L}{\partial z} = 2(z - y)$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} = 2(z - y) x$$

## Vector / matrix case

### Forward pass

$$z = Wx + b$$

$$L = \|z - y\|^2$$

$$W \in \mathbb{R}^{D_{out} \times D_{in}}, x \in \mathbb{R}^{D_{in}}$$

### Backward pass

(chain rule)

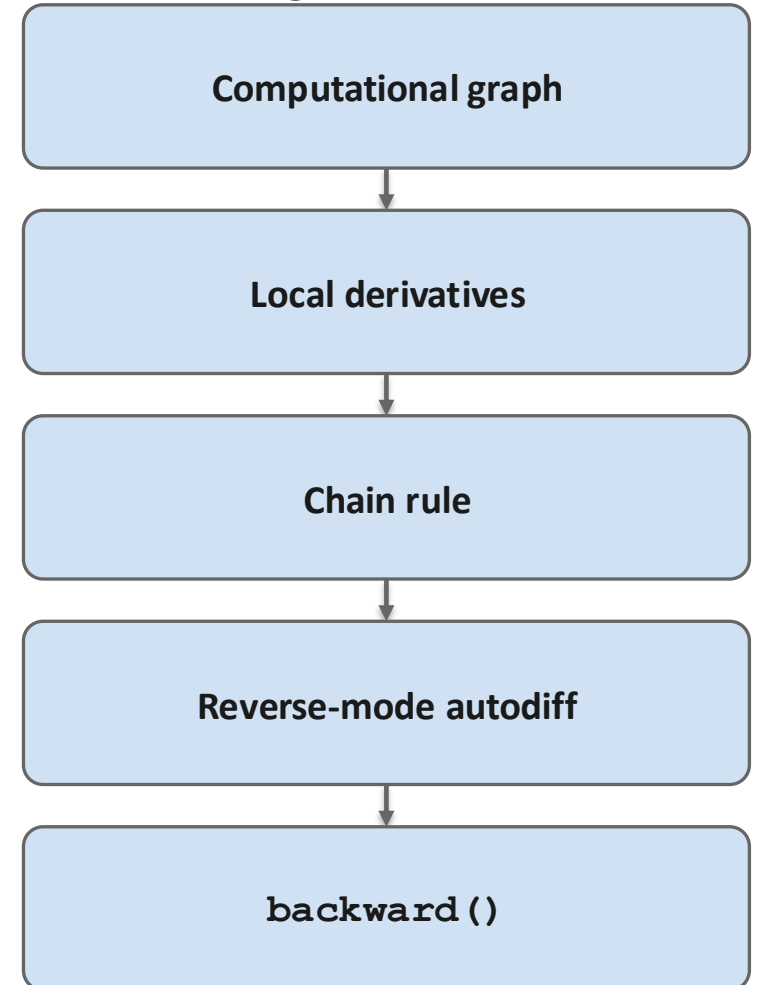
$$\nabla_z L = 2(z - y) \quad \text{shape: } D_{out}$$

$$\nabla_W L = (z - y) x^\top$$

(outer product)

$$\nabla_{w_{ij}} L = \nabla_{z_i} \cdot \nabla_{w_{ij}} z_i$$

## How it fits together



# PyTorch Autograd vs. TensorFlow Eager Autograd

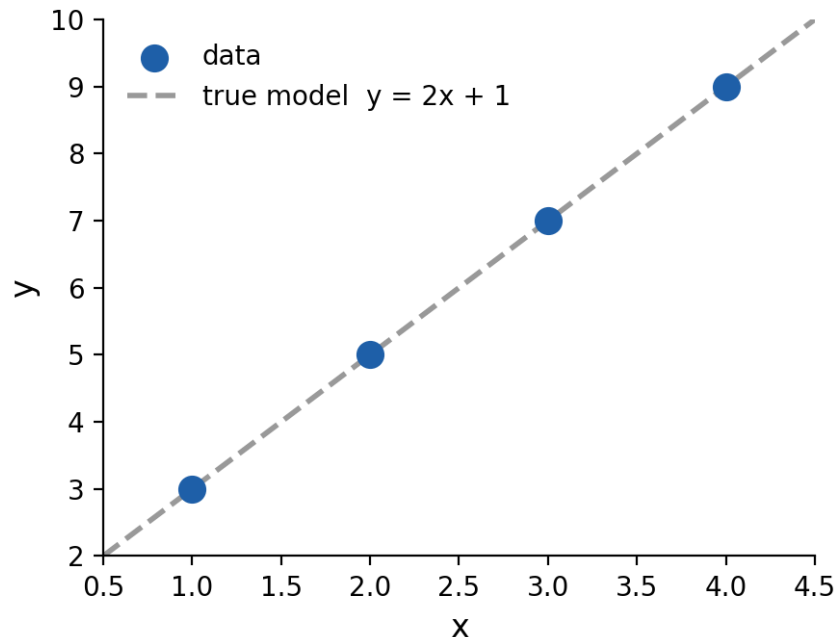
*Same toy regression problem, run through both frameworks' gradient machinery*

## The running example

Fit  $y = wx + b$  to four points:

$x = [1, 2, 3, 4]$ ,  $y = [3, 5, 7, 9]$

(true model:  $y = 2x + 1$ )



## What we'll compare

How gradient tracking is turned on and scoped

Where gradients live once computed

How parameters get updated

The manual loop vs. a built-in optimizer

That both frameworks land on the exact same numbers

# The Manual Training Loop

*Same forward pass and update rule — different gradient mechanics*

## PyTorch

```
w = torch.tensor(0., requires_grad=True)
b = torch.tensor(0., requires_grad=True)

for step in range(5):
    y_pred = w * x + b
    loss = torch.mean((y_pred - y) ** 2)

    loss.backward()

    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad

    w.grad.zero_()
    b.grad.zero_()
```

## TensorFlow Eager

```
w = tf.Variable(0.)
b = tf.Variable(0.)

for step in range(5):
    with tf.GradientTape() as tape:
        y_pred = w * x + b
        loss = tf.reduce_mean((y_pred - y) ** 2)

    dw, db = tape.gradient(loss, [w, b])

    w.assign_sub(learning_rate * dw)
    b.assign_sub(learning_rate * db)
```

*Highlighted: PyTorch accumulates gradients in .grad and must zero them each step; TensorFlow's tape produces fresh gradients with nothing to reset.*

# Swapping in a Built-in Optimizer

*The autograd calls stay the same — only the update step changes*

## PyTorch

```
optimizer = torch.optim.SGD([w, b], lr=0.01)

for step in range(5):
    y_pred = w * x + b
    loss = torch.mean((y_pred - y) ** 2)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

`optimizer.zero_grad()` is still required — gradients still accumulate in `.grad` under the hood.

## TensorFlow Eager

```
optimizer = tf.keras.optimizers.SGD(
    learning_rate=0.01)

for step in range(5):
    with tf.GradientTape() as tape:
        y_pred = w * x + b
        loss = tf.reduce_mean((y_pred - y) ** 2)

    gradients = tape.gradient(loss, [w, b])
    optimizer.apply_gradients(
        zip(gradients, [w, b]))
```

No zeroing needed — `apply_gradients()` consumes the fresh (gradient, variable) pairs from this step's tape and discards them.

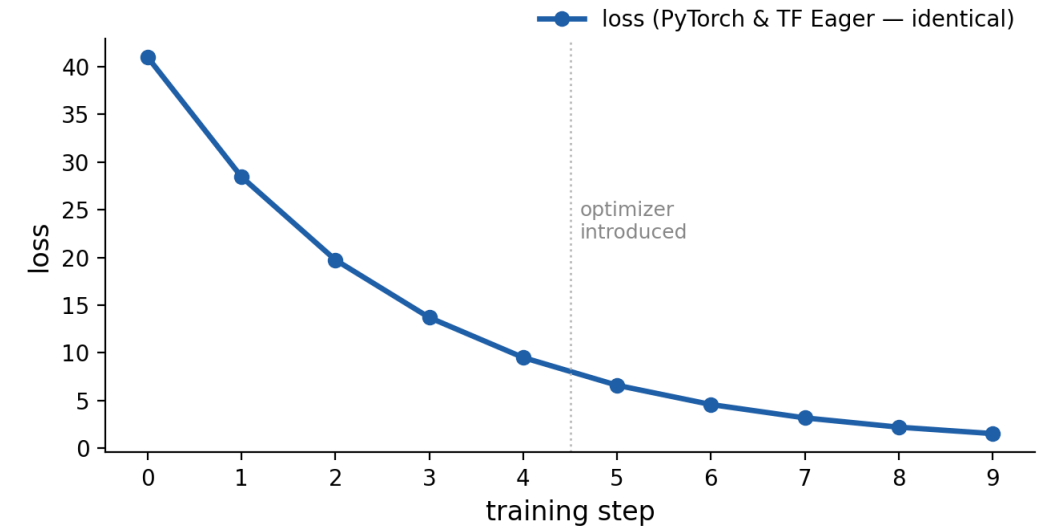
# Same Math, Different APIs

*Both frameworks converge to identical loss,  $w$ , and  $b$  at every step*

## Key differences

| Aspect               | PyTorch                                                                   | TensorFlow Eager                                                         |
|----------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Gradient tracking    | Always on for tensors with <code>requires_grad=True</code>                | Opt-in, scoped to a <code>tf.GradientTape()</code> block                 |
| Where gradients live | Accumulate in the <code>.grad</code> attribute                            | Returned directly by <code>tape.gradient()</code> , not stored           |
| Clearing gradients   | Manual — <code>.zero_()</code> , or <code>optimizer.zero_grad()</code>    | Not needed — the tape is discarded after each step                       |
| Updating parameters  | In-place op inside <code>torch.no_grad()</code> , or <code>.step()</code> | <code>.assign_sub()</code> , or <code>optimizer.apply_gradients()</code> |

## Proof: identical convergence



Both training loops start from  $w=0$ ,  $b=0$  and use the same learning rate. Because they compute the exact same chain-rule derivatives, loss,  $w$ , and  $b$  match to four decimal places at every single step — only the code differs.

# **PYTORCH NN MODULE**

# PyTorch: nn

Higher-level wrapper for  
working with neural nets

Similar to Keras and friends ...  
but only one, and it's good =)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

# PyTorch: nn

Define our model as a  
sequence of layers

nn also defines common ~~loss~~  
functions

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)
```

```
model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)
```

```
learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

# PyTorch: nn

Forward pass: feed data  
to model, and prediction to loss  
function

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

# PyTorch: nn

Backward pass:  
compute all gradients

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad.data
```

# PyTorch: nn

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    model.zero_grad()
    loss.backward()
```

Make gradient step on  
each model parameter



```
for param in model.parameters():
    param.data -= learning_rate * param.grad.data
```

# PyTorch: optim

Use an **optimizer** for different update rules

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
optimizer = torch.optim.Adam(model.parameters(),
                               lr=learning_rate)

for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```

# PyTorch: optim

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out))
loss_fn = torch.nn.MSELoss(size_average=False)

learning_rate = 1e-4
optimizer = torch.optim.Adam(model.parameters(),
                              lr=learning_rate)

for t in range(500):
    y_pred = model(x)
    loss = loss_fn(y_pred, y)

    optimizer.zero_grad()
    loss.backward()

    optimizer.step()
```

Update all parameters  
after computing gradients



# PyTorch: nn

## Define new Modules

A PyTorch **Module** is a neural net layer; it inputs and outputs Variables

Modules can contain weights (as Variables) or other Modules

You can define your own Modules using autograd!

```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

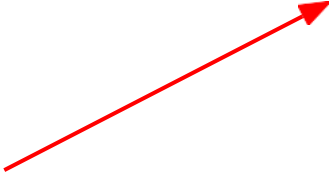
criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

# PyTorch: nn

## Define new Modules

Define our whole model  
as a single Module



```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(1, D_in))
y = Variable(torch.randn(1, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

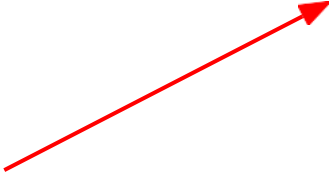
criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

# PyTorch: nn

## Define new Modules

Initializer sets up two children (Modules can contain modules)



```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

# PyTorch: nn

## Define new Modules

Define forward pass using  
child modules and  
autograd ops on Variables

No need to define  
backward - autograd will  
handle it

```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

# PyTorch: nn

## Define new Modules

Construct and train an instance of our model



```
import torch
from torch.autograd import Variable

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, H, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)

    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10

x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

# PyTorch: DataLoaders

A **DataLoader** wraps a **Dataset** and provides minibatching, shuffling, multithreading, for you

When you need to load custom data, just write your own Dataset class

```
import torch
from torch.autograd import Variable
from torch.utils.data import TensorDataset, DataLoader

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

loader = DataLoader(TensorDataset(x, y), batch_size=8)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for epoch in range(10):
    for x_batch, y_batch in loader:
        x_var, y_var = Variable(x), Variable(y)
        y_pred = model(x_var)
        loss = criterion(y_pred, y_var)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

# PyTorch: DataLoaders

Iterate over loader to form minibatches

Loader gives Tensors so you need to wrap in Variables

```
import torch
from torch.autograd import Variable
from torch.utils.data import TensorDataset, DataLoader

N, D_in, H, D_out = 64, 1000, 100, 10

x = torch.randn(N, D_in)
y = torch.randn(N, D_out)

loader = DataLoader(TensorDataset(x, y), batch_size=8)

model = TwoLayerNet(D_in, H, D_out)

criterion = torch.nn.MSELoss(size_average=False)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
for epoch in range(10):
    for x_batch, y_batch in loader:
        x_var, y_var = Variable(x), Variable(y)
        y_pred = model(x_var)
        loss = criterion(y_pred, y_var)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```



# PyTorch: Pretrained Models

Super easy to use pretrained models with  
torchvision <https://github.com/pytorch/vision>

```
import torch
import torchvision

alexnet = torchvision.models.alexnet(pretrained=True)
vgg16 = torchvision.models.vgg16(pretrained=True)
resnet101 = torchvision.models.resnet101(pretrained=True)
```

# Static vs Dynamic Graphs

# Static vs Dynamic Graphs

**TensorFlow:** Build graph once, then run many times (**static**)

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N, D))
y = tf.placeholder(tf.float32, shape=(N, D))
w1 = tf.Variable(tf.random_normal((D, H)))
w2 = tf.Variable(tf.random_normal((H, D)))

h = tf.maximum(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff ** 2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])

learning_rate = 1e-5
new_w1 = w1.assign(w1 - learning_rate * grad_w1)
new_w2 = w2.assign(w2 - learning_rate * grad_w2)
updates = tf.group(new_w1, new_w2)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {x: np.random.randn(N, D),
              y: np.random.randn(N, D),}
    losses = []
    for t in range(50):
        loss_val, _ = sess.run([loss, updates],
                               feed_dict=values)
```

Build graph

Run each iteration

**PyTorch:** Each forward pass defines a new graph (**dynamic**)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

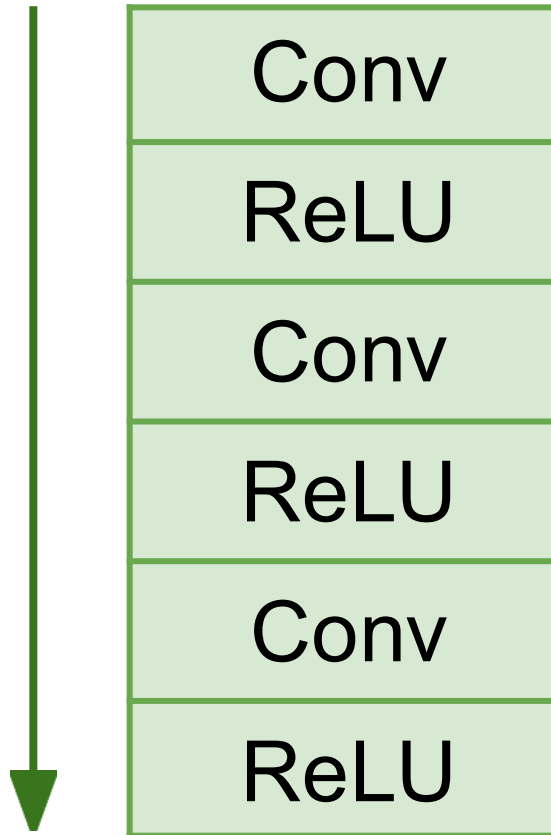
    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

New graph each iteration

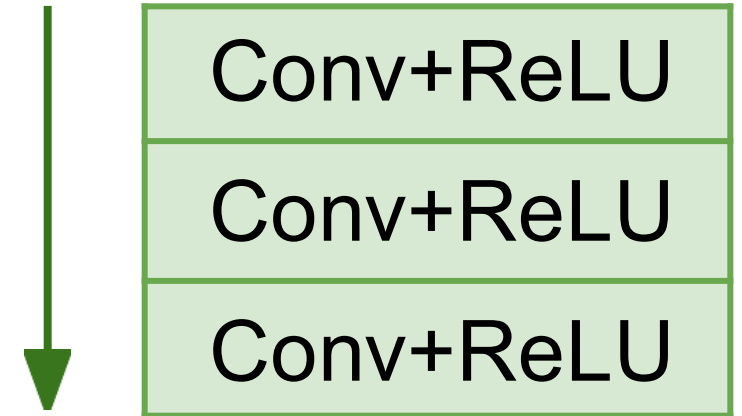
# Static vs Dynamic: Optimization

With static graphs, framework can **optimize** the graph for you before it runs!

The graph you wrote



Equivalent graph with **fused operations**



# Static vs Dynamic: Serialization

## Static

Once graph is built,  
can **serialize** it and  
run it without the code  
that built the graph!

## Dynamic

Graph building and  
execution are intertwined,  
so always need to keep  
code around

Tensorboard

# Visualizing pytorch graphs

```
for t in range(500):
    y_pred = model(x)
    loss = criterion(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

x_input = Variable(torch.randn(1, D_in))

writer = SummaryWriter('runs/exp1')

writer.add_graph(model, x_input)
writer.close()
```

```
import torch
from torch.autograd import Variable
from torch.utils.tensorboard import SummaryWriter

class TwoLayerNet(torch.nn.Module):
    def __init__(self, D_in, D_out):
        super(TwoLayerNet, self).__init__()
        self.linear1 = torch.nn.Linear(D_in, H)
        self.linear2 = torch.nn.Linear(H, D_out)
    def forward(self, x):
        h_relu = self.linear1(x).clamp(min=0)
        y_pred = self.linear2(h_relu)
        return y_pred

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in))
y = Variable(torch.randn(N, D_out), requires_grad=False)
model = TwoLayerNet(D_in, D_out)
optimizer = torch.optim.SGD(model.parameters(), lr=1e-4)
criterion = torch.nn.MSELoss(reduction='mean')
```

# Visualizing pytorch graphs

TensorBoard

GRAPHS

UPLOAD

Search nodes (regex)

Fit to screen

Download PNG

Upload file

Run (1) exp1

Tag (2) Default

Graph type

☒ Op graph

☐ Conceptual graph

☐ Profile

Legend

colors

same substructure

unique substructure

(\* = expandable)

Namespace\* ?

OpNode ?

Unconnected series\* ?

Connected series\* ?

Constant ?

Summary ?

Dataflow edge ?

Control dependency edge ?

TwoLayerNet

Subgraph: 7 nodes

# **MODEL DEPLOUMENT USING ONNX**

# Saving a PyTorch Model: What Actually Gets Saved?

*The learned numbers, not the code — and that distinction matters*

## What's saved: the state\_dict

```
torch.save(model.state_dict(), "model.pth")

# to reload later:
model = LinearModel()
model.load_state_dict(torch.load("model.pth"))
```

A state\_dict is just an OrderedDict of tensor names → learned values. It has no architecture info — you must already have the LinearModel class to load it back into.

## Actual output from our example

```
linear.weight  tensor([[1.9505]])
linear.bias    tensor([1.1456])
```

## Two ways to save — and the trade-off

### state\_dict (recommended)

Just tensors. Robust across code refactors. Needs the class definition available to reload.

### Whole model (torch.save(model, ...))

Pickles the class too. Convenient, but brittle — breaks if the class definition moves or changes.

*For deployment outside the original Python/PyTorch environment, neither option is quite enough — that's what ONNX export solves next.*

# Converting to ONNX: What Can Go Wrong

*Tracing captures one concrete run — you have to tell it what can vary*

## The export call

```
example_input = torch.randn(1, 1)

torch.onnx.export(
    model, example_input, "linear_model.onnx",
    dynamo=True,
    input_names=["input"],
    output_names=["output"],
    dynamic_shapes={"x": {0: "batch_size"}}
)
```

## What each highlighted piece fixes

- `dynamo=True` — modern, more robust exporter (traces via `torch.export`)
- `dynamic_shapes` — keeps batch size symbolic instead of hard-coding it

## Issue we actually hit, running this code

```
ModuleNotFoundError:
  No module named 'onnxscript'
```

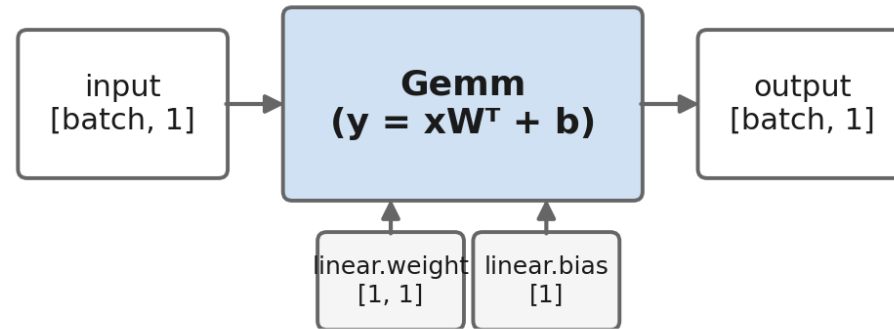
**Fix:** `pip install onnxscript`

## Other issues to watch for (general)

- Dynamic control flow (if/for on tensor values) may get "baked in" as constants unless captured, not just traced
- `example_input`'s shape/dtype becomes the template — mismatches later cause silent or hard errors
- Some ops have no ONNX equivalent at your target opset version

# Inside the ONNX File

*A protobuf graph: inputs, outputs, weights, and operators*



## Validate: structural correctness

```
onnx_model = onnx.load("linear_model.onnx")
onnx.checker.check_model(onnx_model)
```

*# → ONNX model is valid.*

*This checks shapes, types, and graph well-formedness — not numerical correctness. That's a separate check (next slide).*

## Inspect: what our model actually contains

```
Inputs:      input
Outputs:    output
Initializers: linear.weight [1,1]
                  linear.bias [1]
Operators:  Gemm (nn.Linear → one op)
IR / Opset: 10 / 20
Producer:   pytorch 2.14.0
```

# Running the ONNX Model

*ONNX Runtime speaks NumPy, not torch.Tensor — and it checks dtypes strictly*

## Providing input

```
session = ort.InferenceSession(
    "linear_model.onnx",
    providers=["CPUExecutionProvider"])
```

```
input_name = session.get_inputs()[0].name
```

```
x_numpy = x_test.numpy()    # torch → numpy
session.run([output_name], {input_name: x_numpy})
```

## Real error: wrong dtype

```
# passing float64 instead of float32:
InvalidArgument: Unexpected input
data type. Actual: tensor(double),
expected: tensor(float)
```

*Fix: match the exported dtype exactly — here, float32.*

## Dynamic batching: it just works

Because we declared batch\_size as dynamic at export time, any batch size works at inference — no re-export needed:

| input x (shape (6,1)) | output = 1.95x + 1.15 |
|-----------------------|-----------------------|
| 1.0                   | 3.10                  |
| 2.0                   | 5.05                  |
| 3.0                   | 7.00                  |
| 4.0                   | 8.95                  |
| 5.0                   | 10.90                 |
| 100.0                 | 196.19                |

*Without dynamic\_shapes at export, the batch dimension is baked in as a fixed constant — any other batch size raises a shape-mismatch error.*

# Validation and Performance

*Correct first, fast second — and "fast" depends on the model*

## Correctness: PyTorch vs. ONNX Runtime

```
max_difference = np.max(
    np.abs(pytorch_output - onnx_output))
```

**max difference = 0.0 → SUCCESS**

Exact match here because the model is tiny and the export is lossless FP32 → FP32. Larger models can show small numerical drift from operator-level differences — always check against a tolerance, not exact equality.

## Throughput: 1000 x batch-1000 forward passes

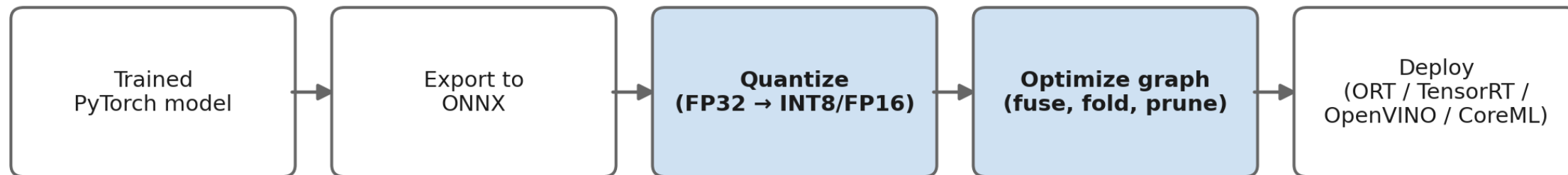
| Runtime         | Inferences / sec |
|-----------------|------------------|
| PyTorch (eager) | ≈ 172,000 / sec  |
| ONNX Runtime    | ≈ 161,000 / sec  |

### Surprise: ONNX Runtime wasn't faster here.

For a single Linear layer, PyTorch's eager-mode overhead is already tiny, so there's little for ONNX Runtime's graph fusion and optimization passes to improve. The gap in ONNX Runtime's favor typically grows with model depth and operator count — always benchmark on your actual model and target hardware, not a rule of thumb.

# Beyond Export: Quantization and Optimization

*Not in the code above — but usually the next step before shipping*



## Quantization: fewer bits per number

### Post-training quantization (PTQ)

Convert FP32 weights/activations to INT8 after training. Fast, some accuracy loss.

### Quantization-aware training (QAT)

Simulate low precision during training. Better accuracy, needs retraining.

## Graph optimization & deployment targets

- ONNX Runtime applies operator fusion, constant folding, and dead-node elimination automatically (graph\_optimization\_level: BASIC / EXTENDED / ALL)
- Same ONNX file can target different backends: TensorRT (NVIDIA GPU), OpenVINO (Intel CPU), CoreML (Apple) — ONNX is the common interchange format
- Always re-validate accuracy after quantizing or optimizing — these are lossy or graph-restructuring transforms, not guaranteed no-ops

*Tools: `onnxruntime.quantization (quantize_dynamic / quantize_static)`,  
`torch.ao.quantization`*

# References

- Deep Learning with Pytorch. Eli Stevens, Luca Antiga, Thomas Viehman, Manning publishers.
- Exporting a model from pytorch to ONNX and running using ONNX runtime:  
[https://pytorch.org/tutorials/advanced/super\\_resolution\\_with\\_onnxruntime.html](https://pytorch.org/tutorials/advanced/super_resolution_with_onnxruntime.html)
- Tensorboard tutorial:  
[https://pytorch.org/tutorials/intermediate/tensorboard\\_tutorial.html](https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html)